

Rigel

From Raw Signals to Pure Clarity

github.com/mahmudmaahi/Rigel

• TEAM ORION · PRESENTERS

MD Mahmudul Hossain Maahi 2305150

Md. Shihabul Hasan 2305139

• PROJECT SUPERVISOR

Md. Roqunuzzaman Sojib

Department of CSE, BUET

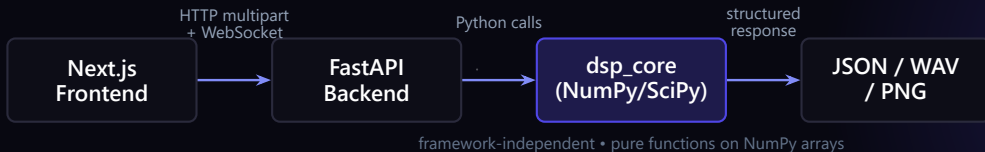
What We'll Cover

- Motivation & problem statement
- System architecture — how the pieces fit together
- A tour of the six DSP workspaces
- Live demonstration on real audio
- Engineering highlights & testing
- Conclusion & future work

Why Rigel?

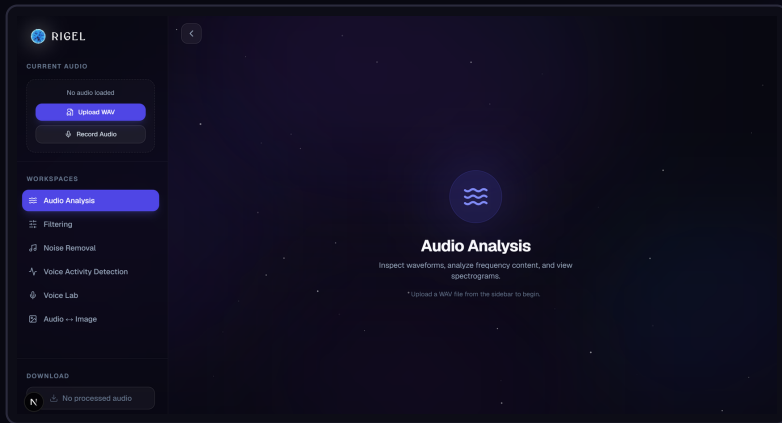
- Most audio software is a **black box** — Rigel exposes every transform it performs
- Built to make classical Signals & Systems theory **tangible**: DFT, STFT, filters, statistical estimation
- One connected platform: analyze, filter, denoise, transform, and losslessly encode audio

Architecture

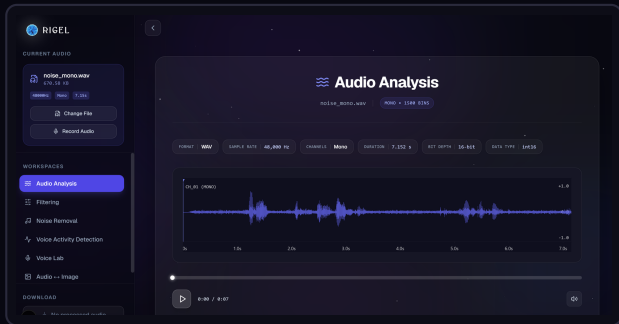


- `dsp_core` never touches HTTP — reusable by any future frontend or a desktop app
- Stack: TypeScript, FastAPI, NumPy, SciPy, raw WebSockets for real-time streams

Six DSP Workspaces



Audio Analysis



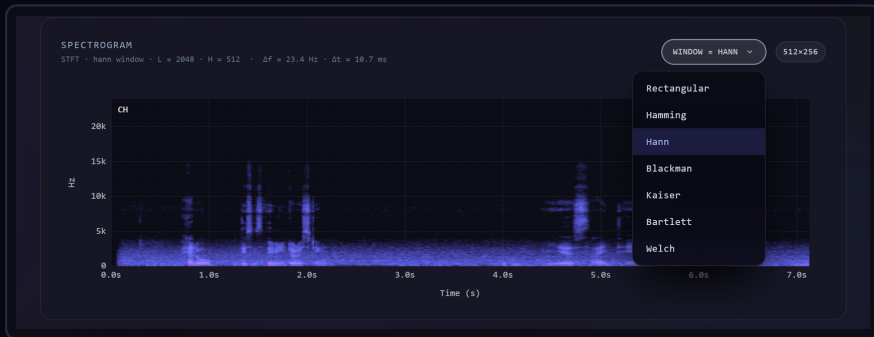
- Peak-envelope (min/max) waveform decimation — transients never disappear when zoomed out
- Full signal metadata: sample rate, bit depth, duration, channel layout

Spectrum & Spectrogram



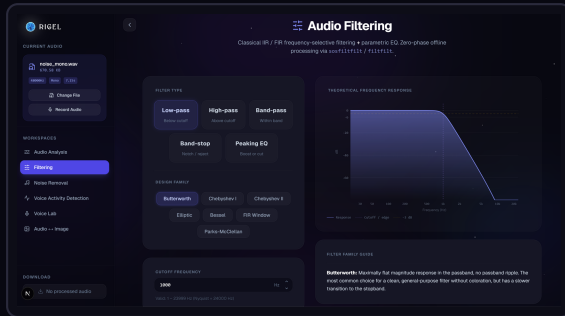
- $X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}$ — an FFT, $O(N \log N)$
- STFT: 2048-sample Hann frames, 512-sample hop (75% overlap)

Choice of Window Matters



- Seven windows: Rectangular, Hamming, Hann, Blackman, Kaiser, Bartlett, Welch
- Trade-off inherent to every window: main-lobe **resolution** vs. side-lobe **leakage**

Frequency-Selective Filtering



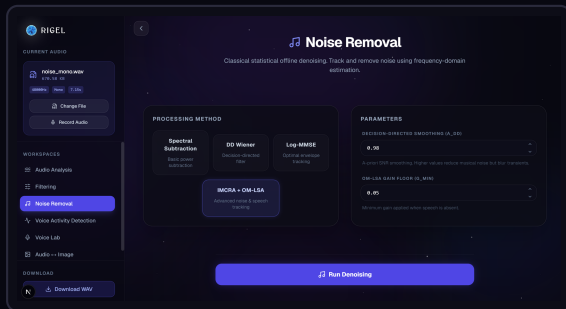
- IIR: Butterworth, Chebyshev I/II, Elliptic, Bessel (stable SOS)
- FIR: window method & Parks–McClellan — zero-phase via `sosfiltfilt`/`filtfilt`

Before & After



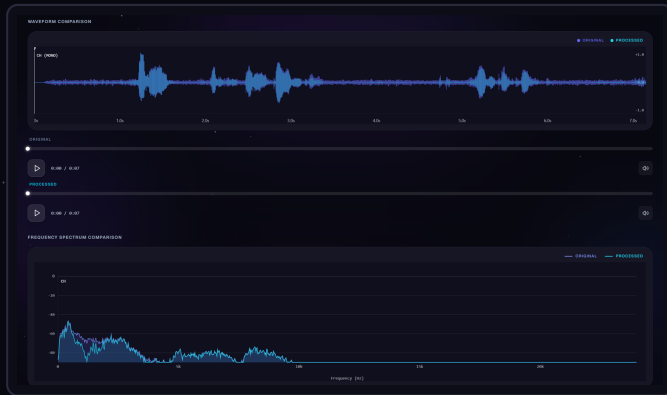
- 1 kHz low-pass: energy above cutoff removed, waveform shape preserved below it

Statistical Denoising



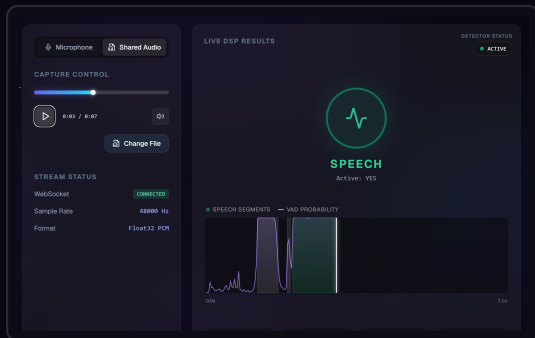
- Spectral Subtraction → Wiener (decision-directed) → Log-MMSE → IMCRA + OM-LSA
- Classical statistical DSP, not deep learning — every gain value is explainable

Denoised Output



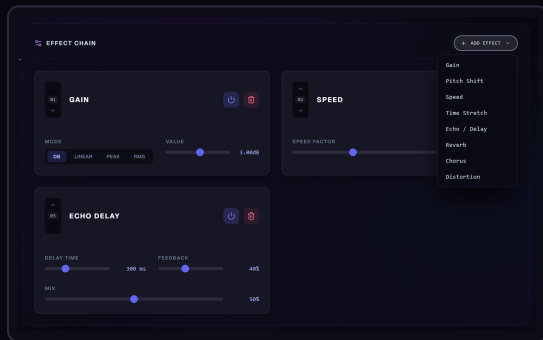
- Broadband noise floor suppressed while speech harmonics are preserved

Real-Time Speech Detection



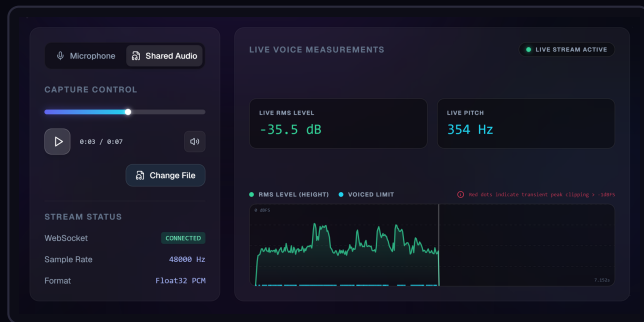
- Classical DSP using Short-time Energy, ZCR and Spectral Features
- Raw Float32 PCM streamed over a binary WebSocket, decision per chunk

Modular Effect Chain



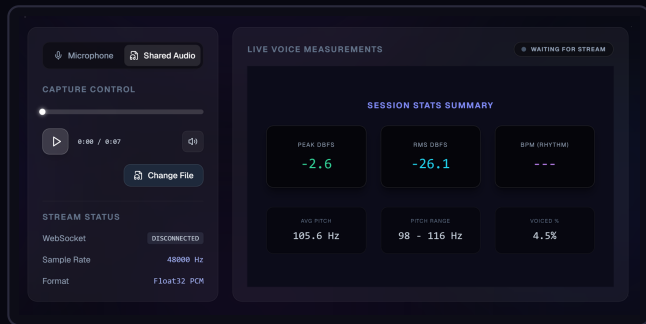
- Deterministic chain: Gain → Speed → Time-Stretch → Pitch Shift → Effect
- 8 effects: Gain, Pitch Shift, Speed, Time Stretch, Echo/Delay, Reverb, Chorus, Distortion

Live Voice Measurement



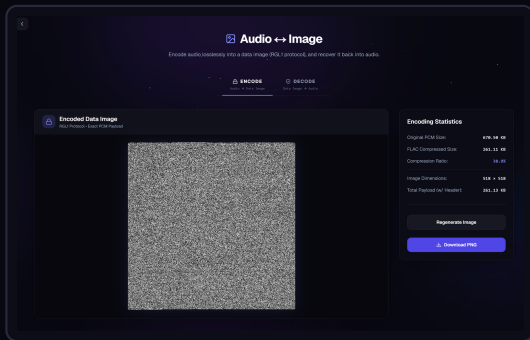
- YIN pitch estimation & RMS loudness, computed per streamed chunk
- Sample rate negotiated client → server over a JSON WebSocket handshake

Session Analytics



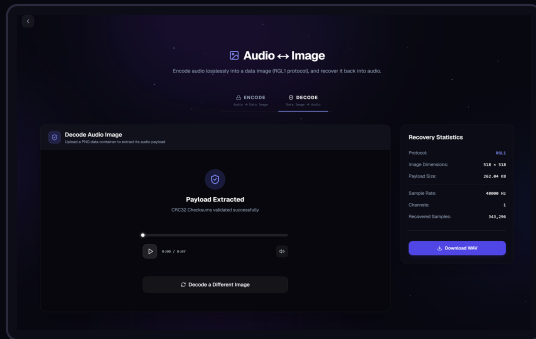
- Peak / RMS loudness, average pitch, pitch range, % voiced frames
- BPM shown only when statistically reliable — never a fabricated number

Encoding Audio as a Data Image



- PCM → FLAC compression → CRC32-checksummed RGL1 packet → PNG pixels
- A literal data container, not steganography — every pixel is payload

Recovering Audio, Bit-Exact



- CRC32-validated on decode — a corrupted image is rejected, never silently guessed

D E M O N S T R A T I O N

Live Demo

noise_mono.wav → Analysis → Denoising → Audio↔Image

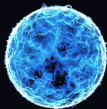
UNDER THE HOOD

Engineering & Testing

- `dsp_core`: pure NumPy/SciPy, zero framework coupling, fully unit-testable
- 480+ backend `pytest` cases spanning all six modules
- Real-time paths: binary WebSocket streaming for VAD and live pitch/RMS
- Zero-phase filtering, CRC32-verified codecs, strict NaN/Inf rejection in effects

Conclusion & Future Work

- Six complete modules — analysis, filtering, denoising, VAD, voice effects, audio-image codec
- A DSP laboratory built to teach, not just to process
- Future Plans: a desktop application, active noise cancellation, learned models



Rigel

Thank You

Questions & Discussion